

Fundamentos Básicos de JAVA

Guía oficial de estudio — Backend 1

¡Hola, futuros desarrolladores! Soy su profesor de Backend 1. En este documento, daremos nuestros primeros pasos en el maravilloso mundo de Java. Iremos paso a paso, asegurándonos de que cada concepto quede claro antes de avanzar.

Objetivo

Comprender los pilares esenciales de la programación en Java, desde cómo guardar un simple dato en memoria hasta cómo estructurar un programa básico usando clases y objetos. Esto formará la base sólida que necesitan para convertirse en excelentes desarrolladores Backend.

1. Variables

¿**Qué son?** Imagina una variable como una caja con una etiqueta donde puedes guardar un dato. Esta "caja" vive en la memoria de la computadora mientras el programa se ejecuta.

Ejemplo:

```
// Declaramos una variable llamada "edad" y le guardamos el número 25
int edad = 25;
```

Ejercicio simple: Imagina que quieres guardar tu edad y tu peso. ¿Cómo llamarías a esas variables? Escríbelo en un papel.

Solución práctica: Usaremos la convención camelCase (empezar en minúscula y cada palabra nueva con mayúscula).

```
int miEdad = 28;
double miPeso = 72.5;

System.out.println("Tengo " + miEdad + " años y peso " + miPeso + " kg.");
```

2. Tipos de variables: El Catálogo Completo

¿Qué son? (Definición profunda) Java es un lenguaje **fuertemente tipado**. Esto significa que no puedes simplemente crear una "caja" genérica; debes decirle al compilador exactamente qué forma y tamaño tendrá esa caja antes de usarla. En Java, los tipos de variables se dividen en dos grandes familias: **Primitivos** (guardan el valor real y son súper rápidos) y **De Referencia** (guardan la dirección de memoria donde vive un Objeto).

Los 8 Tipos Primitivos (La base de Java):

• Números Enteros (Sin decimales):

- `byte`: Ocupa 8 bits. Guarda números del -128 al 127. Útil para ahorrar memoria o datos a nivel de bytes.
- `short`: Ocupa 16 bits. Guarda números del -32,768 al 32,767. Poco usado hoy en día.
- `int`: Ocupa 32 bits. Guarda números hasta los 2.14 mil millones. **Es el estándar por defecto** para números enteros.
- `long`: Ocupa 64 bits. Trillones. **Vital en Backend** para IDs de bases de datos o timestamps. *Nota: Agregar 'L' al final.*

• Números de Punto Flotante (Con decimales):

- `float`: Ocupa 32 bits. 6 a 7 dígitos decimales. *Nota: Agregar 'f' al final.*
- `double`: Ocupa 64 bits. 15 dígitos decimales. **Es el estándar por defecto** para cálculos matemáticos con decimales.

• Caracteres y Lógica:

- `char`: Ocupa 16 bits. Un **único** carácter Unicode entre comillas simples (ej. `'A'`).
- `boolean`: Verdadero (`true`) o falso (`false`). Base de la lógica.

El Tipo Especial: String

`String` no es un tipo primitivo, es una Clase (Tipo de Referencia), pero se usa con tanta frecuencia que Java le da trato especial para textos entre comillas dobles (ej. `"Hola"`).

Ejemplo Práctico Completo:

```
// ENTEROS
byte edadPerro = 12;
int visitasWeb = 500000;
long idBaseDeDatos = 9876543210L; // <-- Nota la 'L' al final

// DECIMALES
float porcentaje = 15.5f; // <-- Nota la 'f' al final
double pi = 3.14159265359;

// OTROS
char letraCalificacion = 'A';
boolean servidorEncendido = true;
String mensaje = "Conexión exitosa a la base de datos";
```

Ejercicio simple: Imagina que estás diseñando el Backend de YouTube. Identifica qué tipo exacto de variable usarías para: a) El número total de visualizaciones de "Baby Shark" (más de 10 mil millones). b) Si el video está en estado "Oculto" o "Público". c) La letra inicial del canal.

Solución práctica:

```
// a) long: Un 'int' normal explotaría (llega a 2.14 mil millones). Necesitamos 64 bits.
long visualizacionesTotales = 130000000000L;

// b) boolean: Al ser solo dos estados posibles, es la opción más eficiente.
boolean isOculto = false;

// c) char: Como es solo un carácter, un char es más ligero que un String completo.
char letraInicial = 'Y';

System.out.println("Canal empieza con: " + letraInicial);
System.out.println("Vistas: " + visualizacionesTotales + " | ¿Oculto?: " + isOculto);
```

3. Casting de variables (Conversión de tipos)

¿Qué es? A veces necesitamos meter el contenido de una caja en otra de diferente tipo. El "casting" es forzar a un tipo de dato a convertirse en otro.

Ejemplo:

```
double numeroDecimal = 9.99;
// Convertimos el decimal a entero forzosamente. Se pierde el .99
int numeroEntero = (int) numeroDecimal; // El resultado es 9
```

Ejercicio simple: Si tienes `double peso = 75.8;` y haces `int pesoEntero = (int) peso;`, ¿Qué valor se guardará en `pesoEntero`?

Solución práctica: Se guardará el valor 75, ya que se trunca la parte decimal.

```
double peso = 75.8;
int pesoEntero = (int) peso;

System.out.println("Peso original: " + peso); // Imprime 75.8
System.out.println("Peso casteado: " + pesoEntero); // Imprime 75
```

4. Clases Wrap (Clases Envoltorio): Investigación Profunda

¿Qué son? (Definición y Propósito) En Java, existen los **datos primitivos** (ultrápidos y ligeros) y los **Objetos** (más pesados, con métodos e inteligencia). Las Clases Wrap son el puente entre ambos mundos, tomando un primitivo y envolviéndolo en un Objeto (ej. `int` -> `Integer`, `double` -> `Double`).

Alcance y Casos de uso en Backend:

1. **Colecciones:** Las listas avanzadas (como `ArrayList`) no aceptan tipos primitivos; exigen objetos `Integer`.
2. **Valores Nulos:** Un primitivo `int` siempre vale `0`, pero un objeto `Integer` puede ser `null` para representar ausencia de datos en bases de datos.
3. **Métodos de Utilidad:** Proveen funciones estáticas para parseo y límites numéricos.

Ejemplo Práctico Ampliado:

```
// 1. Autoboxing: El primitivo 5 se "envuelve" automáticamente
Integer numeroObjeto = 5;

// 2. Unboxing: El objeto se "desenvuelve" para sumarle 10
int suma = numeroObjeto + 10;

// 3. Alcance de Nulos
Integer edadUsuario = null; // Válido si el dato no existe

// 4. Utilidades de conversión (Parsing)
String textoDeFormulario = "150";
int convertido = Integer.parseInt(textoDeFormulario);
```

Ejercicio simple: ¿Qué pasaría al hacer Unboxing de un `Integer` vacío (`null`) guardándolo en un `int` primitivo?

Solución práctica: Lanza un error `NullPointerException`. Siempre debemos validar:

```
Integer cajaVacía = null;

if (cajaVacía != null) {
    int valorPrimitivo = cajaVacía; // Unboxing seguro
    System.out.println("El valor es: " + valorPrimitivo);
} else {
    System.out.println("Atención: El dato venía vacío (null).");
}
```

5. Parsing (Análisis/Conversión desde Texto)

¿Qué es? El proceso de convertir un String (ej. ingresado en un formulario web) a un tipo numérico usando Clases Wrap.

Ejemplo:

```
String edadTexto = "30";  
int edadReal = Integer.parseInt(edadTexto);
```

Ejercicio simple / Solución práctica: Convertir el texto "3.14" a double.

```
String piTexto = "3.14";  
double pi = Double.parseDouble(piTexto);  
double resultado = pi * 2;  
System.out.println("El doble de Pi es: " + resultado);
```

6. Operadores y Tipos

¿Qué son? Símbolos para cálculos y comparaciones: **Aritméticos** (+, -, *, /, %), **Relacionales** (==, !=, >, <, >=), **Lógicos** (&&, ||, !).

Ejercicio simple: ¿Cuál es el resultado de (5 == 5) && (3 > 10)?

Solución práctica: Es **Falso**, ya que el operador && exige ambas partes verdaderas.

```
boolean condicion1 = (5 == 5); // true  
boolean condicion2 = (3 > 10); // false  
boolean resultadoFinal = condicion1 && condicion2;  
  
System.out.println("¿Es verdadero el resultado final?: " + resultadoFinal); // false
```

7. Estructuras de Control

¿Qué son? Señales de tránsito para tomar decisiones o repetir tareas: if / else, switch, while, do while, for, foreach.

Ejercicio / Solución práctica: Ciclo while del 1 al 5.

```
int contador = 1;

while (contador <= 5) {
    System.out.println("Número: " + contador);
    contador++; // Crucial para evitar bucle infinito
}
```

8. Palabras Reservadas

¿Qué son? Palabras que Java reserva para su uso (ej. class, public, static, void, if). No pueden usarse como nombres de variables.

Solución práctica: Si necesitamos contexto similar, agregamos sufijos:

```
int miIf = 5; // Válido
System.out.println("Variable válida: " + miIf);
```

9. Array (Vectores y Matrices)

¿Qué son? Estructuras estáticas para almacenar múltiples elementos del mismo tipo. Vectores (1D) o Matrices (2D).

Ejercicio / Solución práctica: Acceder al número 30 en `int[] edades = {20, 25, 30};`.

```
int[] edades = {20, 25, 30};
int laEdadMayor = edades[2]; // Los índices empiezan en 0
System.out.println("La edad extraída es: " + laEdadMayor);
```

10. Funciones (void y con argumentos)

¿Qué son? Bloques de código con una tarea específica. void indica que no devuelven valor.

Ejercicio / Solución práctica: Función void para sumar dos números.

```
public void imprimirSuma(int a, int b) {  
    int resultado = a + b;  
    System.out.println("El resultado de la suma es: " + resultado);  
}
```

11. ¿Qué son los algoritmos y sus tipos?

¿Qué son? Serie de pasos lógicos y ordenados para resolver un problema (Cualitativos o Cuantitativos).

Solución práctica: Algoritmo de un sándwich traducido a código:

```
public void prepararSandwich() {  
    System.out.println("Paso 1: Tomar dos rebanadas de pan.");  
    System.out.println("Paso 2: Untar mayonesa en una rebanada.");  
    System.out.println("Paso 3: Colocar jamón y queso.");  
    System.out.println("Paso 4: Cubrir con la otra rebanada.");  
    System.out.println("Resultado: ¡Sándwich listo!");  
}
```

12. ¿Qué es el estándar UML?

¿Qué son? Estándar visual de esquemas para planificar sistemas. Los diagramas de clases son clave en Backend.

Solución práctica: Traduciendo un plano UML de una clase "Casa":

```
public class Casa {  
    String color;  
    int numeroPuertas;  
  
    public void abrirPuertaPrincipal() {  
        System.out.println("Puerta principal abierta.");  
    }  
}
```

13. ¿Qué es una Clase?

¿Qué son? Moldes o plantillas que definen atributos y comportamientos de futuros objetos.

Solución práctica: Clase Gato.

```
public class Gato {  
    String nombre;  
    String raza;  
    String color;  
    int edad;  
  
    public void maullar() {  
        System.out.println("¡Miau!");  
    }  
}
```

14. ¿Qué es un Objeto?

¿Qué son? La materialización o instancia de una clase en memoria.

Solución práctica: Instanciando un gato:

```
public class ProgramaPrincipal {  
    public static void main(String[] args) {  
        Gato miPrimerGato = new Gato();  
        miPrimerGato.nombre = "Garfield";  
        miPrimerGato.raza = "Persa";  
        miPrimerGato.color = "Naranja";  
        miPrimerGato.edad = 5;  
  
        System.out.println("Mi gato se llama: " + miPrimerGato.nombre);  
        miPrimerGato.maullar();  
    }  
}
```

*¡Felicidades! Has repasado los fundamentos más críticos de Java y resuelto los ejercicios con código 100% práctico.
¡Sigue practicando!*